

Laborator 2 Arhitecturi Hardware Reconfigurabile

Proiectarea unor structuri logice hardware utilizând circuite reconfigurabile static. Tehnici de lucru cu limbaje de descriere hardware

Conținut laborator:

1. **Obiectivul lucrării.**
2. **Noțiuni teoretice:**
 - a. **Structura unei descrieri în VHDL**
 - b. **Descrierea structurală a circuitelor logice în VHDL;**
 - c. **Descrierea comportamentală a circuitelor logice în VHDL;**
3. **Cerințe laborator.**
4. **Temă.**

1. Obiectivul lucrării:

Laboratorul este organizat ca un scurt, dar util, ghid pentru lucrul cu limbajul de descriere hardware VHDL (VHSIC – Very High Scale Integration Circuit – Hardware Description Language). Acesta este utilizat pentru descrierea circuitelor logice în vederea implementării în arii de porți logice programabile. Din categoria ariilor de porți logice programabile moderne fac parte și circuitele FPGA (Field Programmable Gates Array – Matrici de câmpuri de porți programabile). Întreg fluxul de dezvoltare, pornind de la descrierea circuitului până la implementarea sa pe FPGA poate fi privit ca un proces de reconfigurabilitate statică.

Comportamentul sistemului cu circuit FPGA poate fi schimbat complet prin reconfigurarea acestuia cu o altă descriere VHDL. Pe de altă parte, reconfigurabilitatea este una statică deoarece este un proces prin care toate funcțiile curente pe care circuitul FPGA le are sunt suspendate, deci este scos din regimul normal de operare, și, în plus, este un proces care necesită prezența dezvoltatorului, cel care realizează descrierea.

După enumerarea instrucțiunilor în VHDL, în cadrul prezentării teoretice, lucrarea continuă cu prezentarea, sub forma de probleme, a unor descrieri de structuri logice, cu implementare practică.

Utilitatea laboratorului: Departe de a-și propune să prezinte în întregime subtilitățile limbajului de descriere hardware, lucrarea conține, în principiu, toate noțiunile de bază de VHDL care vor fi necesare pentru lucrul în laboratoarele următoare.

Totodată, lucrarea poate fi utilă ca punct de plecare pentru oricine dorește o carieră în domeniul proiectării circuitelor logice digitale utilizând limbaje hardware de descriere.

2. Noțiuni teoretice

a. Structura unei descrieri în VHDL

O descriere a unui circuit logic într-un limbaj de descriere hardware nu poate fi făcută decât atunci când sunt cunoscute foarte clar caracteristicile circuitului care se dorește descris. Deci, este necesară cunoașterea parametrilor circuitului cum ar fi număr

intrări, tip semnale intrare, numărul și tipul semnalelor de ieșire dar și a funcției pe care acesta trebuie să o îndeplinească: structura internă, comportamentul circuitului.

Din acest motiv, orice descriere în VHDL este structurată pe două module: declarația entității și declarația arhitecturii.

Din punct de vedere al entității, circuitului este privit ca o cutie neagra cu porturile de intrare și ieșire specificate, așa cum se poate vedea în exemplul din figura de mai jos (fig. 1).

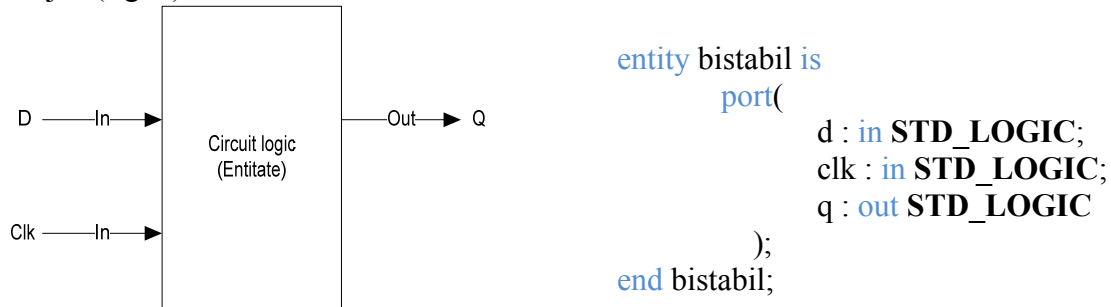


Fig.1. Circuitul logic ca entitate

În entitate sunt declarate porturile de intrare și ieșire. Așa cum se vede în exemplu din figura 1, declarația unui port se face respectând sintaxa:

nume_port : direcționalitate tip_port;

cu următoarele valori prezentate în tabelul 1:

Nume	Descriere	Valori posibile
<i>Nume_port</i>	Numele portului	Poate conține litere, cifre și caracterul _ (underscore). Întotdeauna trebuie să înceapă cu o literă și nu poate conține caracterul dublu ()
<i>Direcționalitate</i>	Direcționalitatea portului	In : port intrare Out : port ieșire Buffer : port ieșire care poate fi conectat ca intrare la un modul intern component al circuitului descris Inout : port care poate fi și ieșire și intrare
<i>Tip_port</i>	Tipul portului declarat	Bit : port pe un bit care poate lua valorile 0 sau 1 Bit_vector (<i>lungime</i> -1 downto 0) : magistrală pe mai mulți biți, de dimensiune <i>lungime</i> – vector de biți Std_logic : port pe un bit care poate lua valorile 0, 1, Z (întă impedență), X (necunoscut), U (nealocat), - (nu contează) etc. conform standardului 1164. Trebuie inclusă librăria 1164

		pentru a putea fi utilizat acest tip. Std_logic_vector (<i>lungime</i>-1 downto 0) : magistrală de semnale std_logic de dimensiunea <i>lungime</i>. Trebuie inclusă librăria 1164 pentru a putea fi utilizat acest tip. Positive : port de tip număr natural. Integer: port de tip număr întreg. Boolean: port de tip boolean. Poate avea valorile true (adevărat) sau false (fals).
--	--	---

Arhitectura conține descrierea circuitului ca schemă de componente constitutive sau ca funcționalitate.

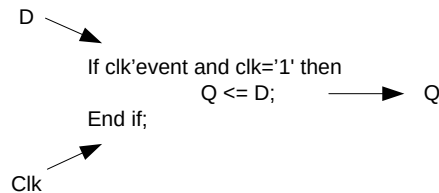


Fig.2. Circuitul logic ca arhitectură

Descrierea funcționalității circuitului în arhitectură se poate face în mai multe feluri așa cum se va prezenta în paragrafele următoare. În general, structura unei arhitecturi este declarată astfel:

```

architecture nume_arhitectură of nume_entitate_asociată is
  --declarații semnale
  Signal nume_semnal : tip_semnal;
  ...
  -- declarații componente
  ...
  -- declarații tipuri de date
  ...
begin
  --corp arhitectură
end nume_arhitectură;
  
```

Semnalele, în terminologia VHDL, reprezintă căile interne de transfer al informației între modulele componente ale unui circuit descris. Numele semnalului și tipul semnalului trebuie să îndeplinească aceleași cerințe ca la definirea numelui și tipului portului din entitate. Componentele nu sunt neapărat prezente în orice descriere VHDL. Asupra modului de declarare al acestora se va reveni în paragraful următor. Dacă se dorește, pot fi create propriile tipuri de semnale. Declarația unui tip nou de semnal are următoarea sintaxă:

type nume_tip_semnal is tip_semnal_asociat;

Tip semnal asociat trebuie să fie unul din tipurile predefinite în librăriile VHDL sau o matrice de semnale.

Există mai multe metode prin care poate fi descrisă funcționalitatea circuitelor logice în VHDL: descrierea structurală și descrierea comportamentală.

b. Descrierea structurală a circuitelor logice în VHDL

Descrierea în VHDL a unui sistem la nivel structural presupune specificarea componentelor din care sistemul este constituit și modul în care acestea sunt interconectate. Componentele pot fi la nivel de porți logice elementare dar pot fi și circuite mai complexe, descrise ulterior în alte porțiuni ale proiectului. Conectările între componente se fac prin intermediul instrucțiunilor de atribuire și prin hărți de porturi în cadrul apelului unei componente.

a.1. Atribuirea semnalelor

Atribuirea semnalelor se face în limbajul VHDL prin simbolul $\leq$ de forma următoare:

Semnal_destinație <= Semnal_sursă;

Exemplu:

a <= b or c;

Semnificația instrucțiunii de mai sus este următoarea: *Semnal_destinație* este conectat la *Semnal_sursă*. Semnalul destinație este întotdeauna unul singur în timp ce semnalul sursă poate fi un singur semnal sau mai multe aflate într-o anumită relație (logică, aritmetică). Deci se poate vorbi de o conectare de tipul rezultat, și anume: rezultatul unei operații dintre mai multe semnale este conectat la semnalul destinație.

O condiție esențială la atribuire este aceea ca dimensiunea semnalelor din ambele părți ale operatorului să fie aceeași.

1.1. Operații logice

Fiecare instrucțiune din VHDL reprezintă de fapt descrierea unui circuit. Pentru circuitele porți logice elementare există cuvinte cheie alocate în VHDL. Astfel:

- SI logic se descrie utilizând cuvântul cheie *and*.
Exemplu: *t1 <= a and b;*
- SAU logic se descrie utilizând cuvântul cheie *or*.
Exemplu: *co <= t1 or t2;*
- SAU EXCLUSIV logic se descrie utilizând cuvântul cheie *xor*.
Exemplu: *stemp <= a xor b;*
- INVERSAREA sau CC1 se reprezintă utilizând cuvântul cheie *not*.
Exemplu: *b_int <= not b;*

Toate expresiile care în exemplele de mai sus au 2 operanzi pot avea mai mulți pentru a descrie porți logice elementare cu mai mult de 2 intrări. De asemenea se pot declara și expresii logice combinate pentru a desemna mai multe porți logice interconectate.

Exemplu: $s \leq \text{not } (\text{stemp xor ci})$;

1.2. Declararea componentelor

La proiectarea unor sisteme mai complexe se folosește foarte des metoda bottom – up. Aceasta presupune inițial proiectarea circuitelor componente, de complexitate mai mică, urmând apoi integrarea acestora în sistemul complex. În cazul în care circuitul care se dorește inclus într-un sistem este declarat în altă parte a proiectului, este într-o librărie a limbajului VHDL sau într-un set de primitive, se utilizează declarația de componente. Aceasta se face prin cuvântul cheie **component**.

Structura unei declarații de componente este prezentată mai jos:

```
component nume_circuit is  
    port(  
        ... declarații porturi...  
    );  
    generic(  
        ...declarații parametrilor...  
    );  
end component;
```

nume_circuit : reprezintă numele circuitului care se dorește integrat așa cum acesta a fost declarat în entitatea sa.

Exemplu:

```
component poarta_sau is  
    port(  
        intrare : in std_logic_vector(dim-1 downto 0);  
        iesire : out std_logic  
    );  
    generic(  
        dim : positive  
    );  
end poarta_sau;
```

Declarațiile de porturi și de parametrilor trebuie să urmărească exact denumirile care au fost utilizate în entitate.

Declarația unei componente se face după ce a fost precizat numele arhitecturii din care componenta va face parte, dar înainte de a începe corpul propriu zis al arhitecturii (înainte de „begin”).

Apelarea componentei se face în corpul arhitecturii. O componentă se apelează utilizând următorul format:

```
nume_circuit_local : nume_circuit port map (...asocieri de porturi...) generic map  
    (...asocieri de parametrilor...);
```

nume_circuit_local : reprezintă numele care îl va avea circuitul în sistemul în care este integrat.

Pentru un sistem care are mai multe circuite va trebui să se utilizeze denumiri diferite la fiecare. Singura excepție este atunci când se utilizează generatorul ciclic de circuite.

nume_circuit: reprezintă numele circuitului integrat, așa cum acesta a fost declarat în entitatea proprie.

Exemplu:

```
poarta_sau_5 : poarta_sau port map(a_in,b_out) generic map(5);
```

Asocierile de porturi și asocierile de parametri se fac prin listarea semnalelor care se doresc conectate la porturile respective. Dimensiunea și tipul semnalelor conectate la porturi trebuie să fie aceleași cu cele declarate în entitatea componentei.

În cazul în care se dorește integrarea într-un sistem a mai multor circuite de același tip se recomandă utilizarea generatorului ciclic de circuite. Declararea acestuia, tot în interiorul arhitecturii se face în felul următor:

```
nume_grup_circuite : for i in valoare_inicială to valoare_finală generate  
    ...apeluri componente...  
end generate;
```

nume_grup_circuite: reprezintă numele grupului de circuite așa cum acesta va fi recunoscut în sistem.

Exemplu:

```
grup_4_porti : fori in 0 to 3 generate  
    poarta_sau_5 : poarta_sau port map(a_in,b_out(i)) generic map(5);  
end generate;
```

Dacă sunt declarate mai multe grupuri de circuite, fiecare va trebui să aibă propriul său nume, excepție făcând cazul în care un grup de circuite este declarat la rândul lui într-un generator ciclic de circuite.

Contorul de circuite *i* poate lua orice valoare inițială și finală (numere întregi) și poate fi utilizat și ca index de semnal în cazul utilizării unor vectori de semnale în apelul componentelor din interiorul generatorului.

Alături de componente se pot efectua, în generatorul ciclic, operații de atribuire de semnale sau chiar alte generatoare ciclice de circuite.

c. Descrierea comportamentală a circuitelor în VHDL

O altă soluție în descrierea funcțională a circuitelor în limbajul VHDL este descrierea comportamentală în procese. Un proces este un modul care poate fi privit similar cu o componentă de la descrierea structurală. Are propriile sale intrări și ieșiri și mai multe procese rulează în paralel unul față de celălalt, comunicând între ele prin semnale (signal).

Spre deosebire de o componentă, procesul conține în interiorul său un program secvențial care îi descrie comportamentul. Acesta poate avea variabile locale, instrucțiuni decizionale, ciclice și chiar apeluri de funcții. Instrucțiunile se execută secvențial, în sensul în care acestea sunt scrise în interiorul procesului, iar când se ajunge la sfârșit procesul intră într-o stare de așteptare. El se va reactiva la modificarea stării oricărei intrări precizate în lista de senzitivități.

Structura unui proces declarat în VHDL este următoarea:

[etichetă:] process (listă semnale senzitivități)
declarații variabile;
begin

instrucțiuni secvențiale;

end process;

Exemplu:

```
process(clk,d)
variable q_int : std_logic;
begin
    if clk'event and clk='1' then
        q_int := d;
    end if;
end process;
```

Eticheta este opțională și, în cazul în care este declarată, trebuie să fie unică. Restricțiile sintaxei utilizate la declarația etichetei sunt similare cu cele de la etichetele de la componente descrise în laboratorul trecut.

Semnalele care se găsesc pe lista de senzitivității a procesului sunt cele care vor declanșa execuția acestuia și mai sunt cunoscute și ca intrări ale procesului. Procesul reprezintă de fapt o secvență infinită. Starea acestuia alternează între așteptare și executarea instrucțiunilor secvențiale descrise în interiorul lui. Inițial procesul intră într-o stare de așteptare, până când se schimbă starea cel puțin a unuia din semnalele din lista de senzitivități. Din acest moment sunt executate instrucțiunile secvențiale din corpul procesului. La terminarea execuției acestora procesul reintră în starea de așteptare iar ciclul se reia.

Variabilele declarate în interiorul procesului au următoarea structură:

variable *denumire* : *tip variabilă*;

unde *denumire* și *tip* sunt similare cu cele de la declarația semnalelor.

Exemplu:

```
variable q_int : std_logic;
```

O variabilă este vizibilă numai în interiorul procesului. Comunicarea între procese se face prin intermediul semnalelor.

Printre instrucțiunile secvențiale, care pot exista în interiorul unui proces menționăm următoarele:

- Instrucțiunea condiționată:

```
if condiție logică then
    bloc de instrucțiuni 1;
[else
    bloc de instrucțiuni 2;]
end if;
```

Exemplu:

```
if a=1 then
    b_out <= '1';
    c_out <= '0';
else
    b_out <= '0';
    c_out <= '1';
end if;
```

Instrucțiunea este una tipică pentru un limbaj de programare de nivel înalt. Dacă condiția logică (o expresie care are drept rezultat adevărat (true) sau fals (false)) este adevărată atunci se execută blocul de instrucțiuni 1. Opțional se poate trata cazul în care condiția logică nu este adevărată, când se execută cel de-al doilea bloc de instrucțiuni.

Condiția logică poate fi dată de propoziții logice care utilizează operatorii de comparare: =, <, >, <=, >=, <> și propoziții logice compuse utilizând and și or. Tot aici pot fi prezente și atributele. Un atribut destul de des utilizat este atributul *event*. Acesta indică faptul că pe semnalul căruia îi este asociat a apărut o schimbare de stare. De exemplu:

```
If clk'event and clk='1' then
    ...
```

s-ar putea „traduce” în felul următor: „dacă pe semnalul clk a apărut un eveniment (și-a schimbat starea) și semnalul de ceas este acum 1 logic atunci ...”.

- Instrucțiunea condiționată multiplă:

```
case expresie de intrare is
    when valoare 1 expresie de intrare =>
        bloc de instrucțiuni 1;
    when valoare 2 expresie de intrare =>
        bloc de instrucțiuni 2;
    ...
    when others =>
        bloc de instrucțiuni i;
end case;
```

Exemplu:

```
case s is
    when '0' => y0 <= '1'; y1 <= '0';
    when others => y0 <= '0'; y1 <= '1';
end case;
```

Expresia de intrare poate fi o condiție logică dar și o variabilă, un semnal sau un apel de funcții. Instrucțiunea permite testarea mai multor valori ale expresiei de intrare și răspunsul la acestea. Astfel, dacă expresia de intrare are valoarea 1 atunci se execută blocul de instrucțiuni 1, dacă are valoarea 2 se execută blocul de instrucțiuni 2 ș.a.m.d. În mod obligatoriu, trebuie să existe și cazul *others* în care expresia de intrare are altă

valoare decât cele precizate explicit. În acest caz se execută blocul de instrucțiuni i. Blocul i poate lipsii dar nu se poate să nu existe cazul *others* specificat.

De menționat aici ar fi și instrucțiunea condiționată de selecție *with...select* care nu se declară în interiorul unui proces.

Sintaxa acesteia este următoarea:

```
with semnal intrare select  
    semnal ieșire <= valoare 1 semnal ieșire when valoare 1 semnal intrare,  
        valoare 2 semnal ieșire when valoare 2 semnal intrare,  
    ...  
    valoare semnal ieșire i when others;
```

Exemplu:

```
with sel select  
y <= i0 when '0',  
i1 when others;
```

Cele două instrucțiuni sunt asemănătoare dar contextul în care sunt apelate este complet diferit. Așa cum am menționat, această instrucțiune nu se declară în interiorul unui proces. Pentru această instrucțiune este construit un bloc de selecție separat, care va lucra în paralel cu orice eventual proces declarat în corpul arhitecturii. Dacă semnalul de intrare are valoarea 1 atunci pe ieșire este generată valoarea 1 pentru semnalul de ieșire. Dacă semnalul de intrare are valoarea 2 atunci pe ieșire este generată o altă valoare 2 pentru semnalul de ieșire, ș.a.m.d. Dacă semnalul de intrare nu are nici una din valorile de precizate mai sus atunci pe ieșire va fi valoarea i.

- Instrucțiunea de execuție ciclică:

Sintaxa acesteia este următoarea:

```
for nume contor in valoare inițială contor to valoare finală contor  
    [step pas incrementare contor] loop  
    bloc de instrucțiuni;  
end loop;
```

Exemplu:

```
for i in 0 to 9 loop  
    v(i) <= I;  
end loop;
```

Instrucțiunea execută blocul de instrucțiuni de un anumit număr de ori, precizat prin valoarea inițială, valoarea finală și pasul de incrementare al contorului. Contorul, al cărui nume este precizat imediat după **for**, se comportă ca o variabilă locală, vizibilă numai în interiorul instrucțiunii **for**. Nu se declară în secțiunea de variabile, și poate fi utilizat pentru indexarea unor vectori sau în alte operații.

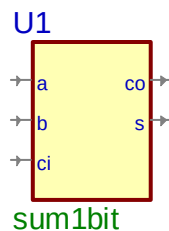
Precizăm că instrucțiunea **for** nu are nici un fel de legătură cu instrucțiunea **for...generate** prezentată în paragraful anterior și utilizată la descrierea structurală.

Cerințe laborator:

1. Să se descrie în limbajul VHDL două circuite: un circuit de adunare a doi operanzi pe 4 biți fiecare și un circuit de scădere pentru aceeași doi operanzi. Operanzii se vor introduce de la butoanele comutatoare (SW_DIP) iar rezultatul este afișat pe display-ul cu led-uri 7 segmente.
1. Proiectarea circuitului de adunare în VHDL.

După crearea unui proiect în ActiveHDL (etape prezentate în anexa care există în același director cu platforma de laborator), pentru proiectarea circuitului de adunare se vor urma pașii:

- a. se va crea un fișier nou, VHDL, utilizând wizard-ul, denumit sum1bit.vhd. Aici se va descrie sumatorul complet pe un bit care va intra în componența atât a circuitului de adunare cât și a celui de scădere. Structura sumatorului pe un bit, la nivel de semnale de intrare și ieșire este prezentată în figura de mai jos.



Circuitul are 3 intrări pe un bit (a,b,ci) și două ieșiri pe un bit (co,s).
Declarația acestuia în VHDL, după construirea fișierului vhd, va fi următoarea:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity sum1bit is
    port(
        a : in STD_LOGIC;
        b : in STD_LOGIC;
        ci : in STD_LOGIC;
        s : out STD_LOGIC;
        co : out STD_LOGIC
    );
end sum1bit;

architecture sum1bit_a of sum1bit is
```

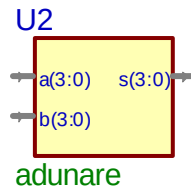
```

signal stemp,t1,t2 : std_logic;
begin
    stemp <= a xor b;
    t1 <= a and b;
    s<= stemp xor ci;
    t2 <= stemp and ci;
    co <= t1 or t2;

end sum1bit_a;

```

- b. se va crea un fișier denumit adunare.vhd. Acesta va reprezenta circuitul de adunare a două numere pe patru biți fiecare. Structura sa este reprezentată în figura de mai jos.



Circuitul are două intrări pe 4 biți fiecare și o ieșire pe 4 biți. Structura sa internă, după crearea fișierului, trebuie să fie următoarea.

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity adunare is
    port(
        a : in STD_LOGIC_VECTOR(3 downto 0);
        b : in STD_LOGIC_VECTOR(3 downto 0);
        s : out STD_LOGIC_VECTOR(3 downto 0)
    );
end adunare;

architecture adunare_a of adunare is
    signal trans : std_logic_vector(4 downto 0);
    component sum1bit is
        port(
            a : in STD_LOGIC;
            b : in STD_LOGIC;
            ci : in STD_LOGIC;
            s : out STD_LOGIC;
            co : out STD_LOGIC
        );
    end component;

```

```
begin
```

```
    sumatoare: for i in 0 to 3 generate
```

```
        suma : sum1bit port map (a(i),b(i),trans(i),s(i),trans(i+1));
```

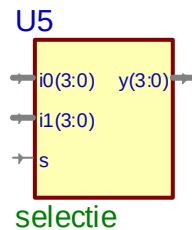
```
    end generate;
```

```
    trans(0) <= '0';
```

```
end adunare_a;
```

2. Proiectarea circuitului de scădere

Pentru circuitul de scădere este necesar proiectarea unui circuit de selecție pentru două numere ep 4 biți fiecare. Circuitul de selecție va fi descris într-un fișier nou creat, denumit selectie.vhd, și are următoarea structură:



selectie

Circuitul are 2 intrări pe 4 biți fiecare, denumite aici i0 și i1 și o intrare de selecție s pe un bit. De asemenea, circuitul are o ieșire pe 4 biți denumită y. Descrierea circuitului este prezentată în listingul de mai jos.

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.all;
```

```
entity selectie is
```

```
    port(
```

```
        s : in STD_LOGIC;
```

```
        i0 : in STD_LOGIC_VECTOR(3 downto 0);
```

```
        i1 : in STD_LOGIC_VECTOR(3 downto 0);
```

```
        y : out STD_LOGIC_VECTOR(3 downto 0)
```

```
    );
```

```
end selectie;
```

```
architecture selectie_a of selectie is
```

```
begin
```

```
    with s select
```

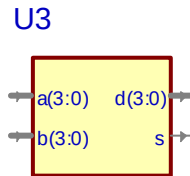
```
    y <= i0 when '0',
```

```
    i1 when '1',
```

```
    "0000" when others;
```

end selectie_a;

Structura circuitului de scădere, creat în fișierul scădere.vhd, este prezentată în figura de mai jos.



scadere

Circuitul are două intrări pe 4 biți fiecare (a și b), o ieșire pe 4 biți (d) și un bit de ieșire pentru semn (s).

Structura circuitului este declarată în listingul de mai jos.

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity scadere is
  port(
    a : in STD_LOGIC_VECTOR(3 downto 0);
    b : in STD_LOGIC_VECTOR(3 downto 0);
    d : out STD_LOGIC_VECTOR(3 downto 0);
    s : out STD_LOGIC
  );
end scadere;

architecture scadere_a of scadere is
  signal a_int, b_int, s_int : std_logic_vector(4 downto 0);
  signal s_intnot, s_int2 : std_logic_vector(3 downto 0);
  signal trans : std_logic_vector(5 downto 0);
  signal trans2 : std_logic_vector(4 downto 0);
  component sum1bit is
    port(
      a : in STD_LOGIC;
      b : in STD_LOGIC;
      ci : in STD_LOGIC;
      s : out STD_LOGIC;
      co : out STD_LOGIC
    );
  end component;
  component selectie is
    port(
      s : in STD_LOGIC;
      i0 : in STD_LOGIC_VECTOR(3 downto 0);

```

```

        il : in STD_LOGIC_VECTOR(3 downto 0);
        y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;
begin

    sumatoare : for i in 0 to 4 generate
        suma : sum1bit port map(a_int(i),b_int(i),trans(i),s_int(i),trans(i+1));
    end generate;

    sumatoare2 : for i in 0 to 3 generate
        suma2 : sum1bit port map('0',s_intnot(i),trans2(i),s_int2(i),trans2(i+1));
    end generate;

    se : selectie port map(s_int(4),s_int(3 downto 0),s_int2(3 downto 0),d);

    trans(0) <= '1';
    trans2(0) <= '1';
    a_int(3 downto 0) <= a;
    a_int(4) <= '0';
    b_int(3 downto 0) <= not b;
    b_int(4) <= '1';
    s <= s_int(4);
    s_intnot <= not s_int(3 downto 0);

end scadere_a;

```

3. Descrierea întregului sistem:

Pentru a descrie întreg sistemul este necesară și proiectarea unui circuit decodor 7 segmente care permite afișarea cifrelor pe display-ul cu LED-uri.

Decodorul 7 segmente se va descrie într-un fișier denumit decodor.vhd. Structura acestuia este prezentată în figura de mai jos.



Circuitul are o intrare pe 4 biți (digital) și o ieșire pe 7 biți (led). Descrierea acestuia în VHDL este următoarea:

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity decodor is

```

```

    port(
        digital : in STD_LOGIC_VECTOR(3 downto 0);
        led : out STD_LOGIC_VECTOR(6 downto 0)
    );
end decodor;

```

architecture decodor_a of decodor is
begin

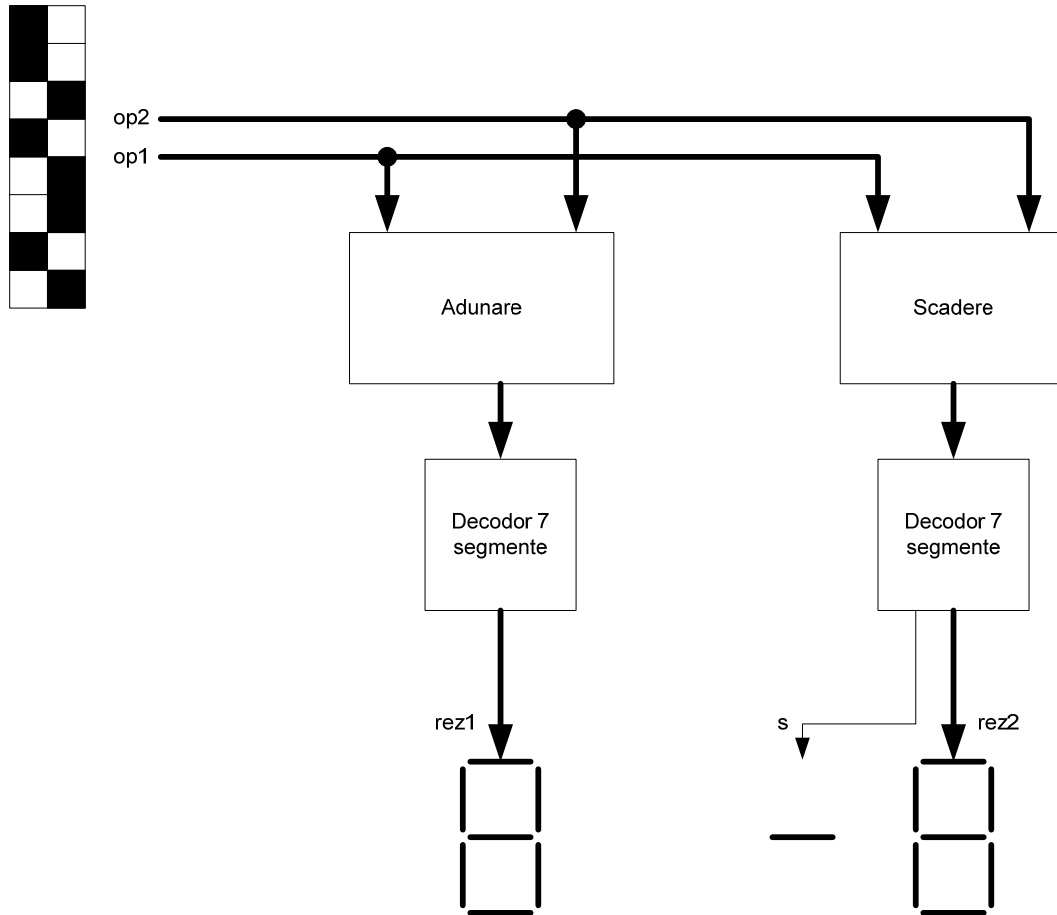
```

    with digital select
    led <= "0111111" when "0000",
        "0000110" when "0001",
        "1011011" when "0010",
        "1001111" when "0011",
        "1100110" when "0100",
        "1101101" when "0101",
        "1111101" when "0110",
        "0000111" when "0111",
        "1111111" when "1000",
        "1101111" when "1001",
        "1110111" when "1010",
        "1111100" when "1011",
        "0111001" when "1100",
        "1011110" when "1101",
        "1111001" when "1110",
        "1110001" when "1111",
        "0000000" when others;

```

end decodor_a;

Structura sistemului care se dorește realizat este prezentată în figura de mai jos. Sistemul va avea două intrări (op1 și op2) pe patru biți fiecare, două ieșiri (rez1 și rez2) pe 7 biți și o ieșire (semn) pe un bit. Intrările vor fi generate de la comutatoarele (SW_DIP) de pe bareta de comutatoare a machetei cu Spartan 3 iar ieșirile vor fi conectate la afișoarele 7 segmente. Sistemul va fi descris în limbajul VHDL într-un fișier nou, intitulat sistem.vhd.



Descrierea în VHDL a sistemului este prezentată în listing-ul de mai jos:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity sistem is
  port(
    op1 : in STD_LOGIC_VECTOR(3 downto 0);
    op2 : in STD_LOGIC_VECTOR(3 downto 0);
    rez1 : out STD_LOGIC_VECTOR(6 downto 0);
    rez2 : out STD_LOGIC_VECTOR(6 downto 0);
    semn : out STD_LOGIC
  );
end sistem;
```

```
architecture sistem_a of sistem is
  signal rez_a, rez_s : std_logic_vector(3 downto 0);
```



```

component adunare is
    port(
        a : in STD_LOGIC_VECTOR(3 downto 0);
        b : in STD_LOGIC_VECTOR(3 downto 0);
        s : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;
component scadere is
    port(
        a : in STD_LOGIC_VECTOR(3 downto 0);
        b : in STD_LOGIC_VECTOR(3 downto 0);
        d : out STD_LOGIC_VECTOR(3 downto 0);
        s : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;
component decodor is
    port(
        digital : in STD_LOGIC_VECTOR(3 downto 0);
        led : out STD_LOGIC_VECTOR(6 downto 0)
    );
end component;
begin

    ad : adunare port map(op1,op2,rez_a);
    sc : scadere port map(op1, op2,rez_s,semn);
    dec1 : decodor port map(rez_a,rez1);
    dec2 : decodor port map(rez_s,rez2);

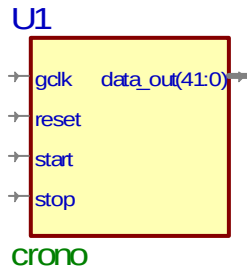
end sistem_a;

```

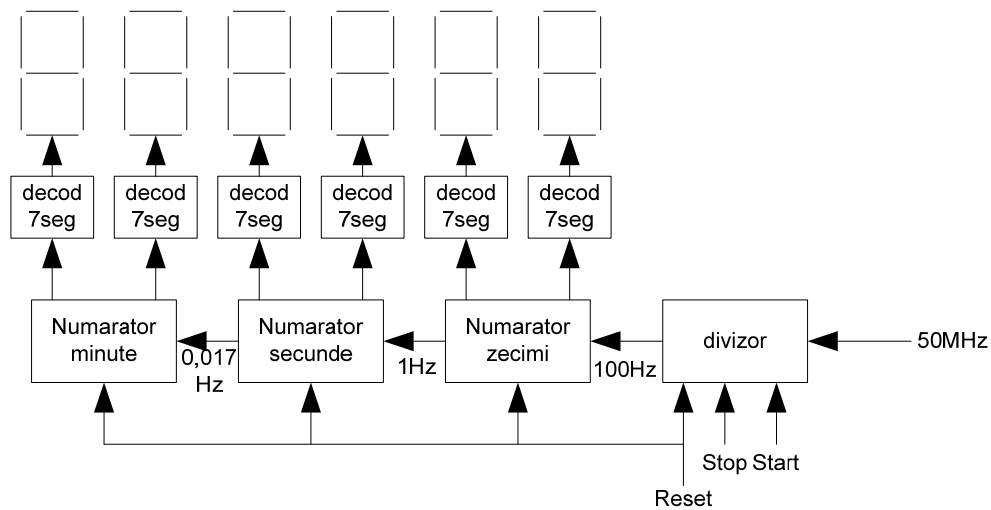
Intrările și ieșirile din sistem se vor asocia cu pinii FPGA care sunt conectați la grupul de comutatoare (SW_DIPx) sau la afișoarele de led-uri 7 segmente (DIGxSEGy). Valorile pinilor sunt date în anexa asociată cu acest laborator, în ultima parte alocată prezentării machetei.

2. Să se construiască un cronometru digital. Acesta va avea ca intrări trei butoane (SW_USER0, SW_USER1 și SW_USER2) unul pentru start, al doilea pentru stop și cel de-al treilea pentru reset. Ieșirile vor fi conectate la 6 afișaje 7 segmente. Pe afișaj se va afișa timpul în formatul mmsszz (mm minute, ss secunde și zz zecimi de secunda). Pe machetă aceste ieșiri sunt notate cu DIGxSEGy unde x reprezintă digitul (0 e cel mai din stânga) și z reprezintă segmentul. Ca intrare referință de timp se va utiliza intrarea de ceas GCLK de la machetă (pentru corespondența pinilor vezi anexa de la laborator).

În figura de mai jos este prezentat sistemul descris ca semnale de intrare și de ieșire.



Schema constructivă sau funcțională a cronometrului este prezentată mai jos:



a. Decodorul 7 segmente (decod7seg.vhd)



```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
```

```
entity decod7seg is
    port(
        binar : in STD_LOGIC_VECTOR(3 downto 0);
        bcd : out STD_LOGIC_VECTOR(6 downto 0)
    );
end decod7seg;
```

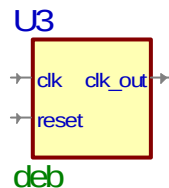
```
architecture decod7seg_a of decod7seg is
begin
```

```

with binar select
bcd <= "0111111" when "0000",
      "0000110" when "0001",
      "1011011" when "0010",
      "1001111" when "0011",
      "1100110" when "0100",
      "1101101" when "0101",
      "1111101" when "0110",
      "0000111" when "0111",
      "1111111" when "1000",
      "1101111" when "1001",
      "0000000" when others;
end decod7seg_a;

```

b. Circuitul pentru debouncing (deb.vhd)



```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity decod7seg is
    port(
        binar : in STD_LOGIC_VECTOR(3 downto 0);
        bcd : out STD_LOGIC_VECTOR(6 downto 0)
    );
end decod7seg;

--}} End of automatically maintained section

```

```

architecture decod7seg_a of decod7seg is
begin

```

```

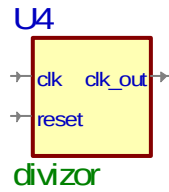
    with binar select
    bcd <= "0111111" when "0000",
          "0000110" when "0001",
          "1011011" when "0010",
          "1001111" when "0011",
          "1100110" when "0100",
          "1101101" when "0101",
          "1111101" when "0110",
          "0000111" when "0111",
          "1111111" when "1000",
          "1101111" when "1001",

```

```

        "0000000" when others;
end decod7seg_a;
c. Divizorul de frecvență (divizor.vhd)

```



```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

```

```

entity divizor is
    port(
        clk : in STD_LOGIC;
        reset : in STD_LOGIC;
        clk_out : out STD_LOGIC
    );
end divizor;

```

```

architecture divizor_a of divizor is
begin

```

```

    process(clk,reset)
        variable cnt : integer;
    begin
        if clk'event and clk='1' then
            if reset = '0' then                -- tasta neapasata
                cnt := 0;
                clk_out <= '0';
            else                                -- tasta apasata
                if cnt = 500000 then
                    clk_out <= '1';
                    cnt := 0;
                else
                    clk_out <= '0';
                    cnt := cnt + 1;
                end if;
            end if;
        end if;
    end process;

```

```

end divizor_a;

```

d. Cronometru

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.all;

entity crono is
    port(
        gclk : in STD_LOGIC;
        reset : in STD_LOGIC;
        start : in STD_LOGIC;
        stop : in STD_LOGIC;
        data_out : out STD_LOGIC_VECTOR(41 downto 0)
    );
end crono;

architecture crono_a of crono is
    --declaratii componente
    component decod7seg is
        port(
            binar : in STD_LOGIC_VECTOR(3 downto 0);
            bcd : out STD_LOGIC_VECTOR(6 downto 0)
        );
    end component;
    component deb is
        port(
            clk : in STD_LOGIC;
            reset : in STD_LOGIC;
            clk_out : out STD_LOGIC
        );
    end component;
    component divizor is
        port(
            clk : in STD_LOGIC;
            reset : in STD_LOGIC;
            clk_out : out STD_LOGIC
        );
    end component;
    --declaratii semnale
    --evenimente taste
    signal ev_start, ev_stop, ev_reset : std_logic;
    --semnal 10ms
    signal div10ms : std_logic;
    --semnal de la sutimi la secunde
    signal sut_sec : std_logic;
    --semnal de la secunde la minute
    signal sec_min : std_logic;
    --semnale iesire numaratoare sutimi, secunde, minute

```

```
signal sutimi, secunde, minute : std_logic_vector(7 downto 0);
```

```
begin
```

```
--evenimente taste
```

```
butstart : deb port map(gclk,start,ev_start);
```

```
butstop : deb port map(gclk,stop,ev_stop);
```

```
butreset : deb port map(gclk,reset,ev_reset);
```

```
--divizor 10 milisecunde
```

```
diviz10ms : divizor port map(gclk,reset,div10ms);
```

```
--numarator sutimi de secunda
```

```
process(div10ms,ev_start,ev_stop,ev_reset)
```

```
variable cnt_unit, cnt_zeci : std_logic_vector(3 downto 0);
```

```
variable c_out : std_logic;
```

```
variable stare_numarare : boolean;
```

```
begin
```

```
--daca este reset apasat reseteaza contorul
```

```
if ev_reset = '1' then
```

```
    cnt_unit := "0000";
```

```
    cnt_zeci := "0000";
```

```
    stare_numarare := false;
```

```
    c_out := '0';
```

```
else
```

```
--daca nu testeaza starea butoanelor de start sau stop
```

```
if ev_start = '1' then
```

```
    stare_numarare := true;
```

```
end if;
```

```
if ev_stop = '1' then
```

```
    stare_numarare := false;
```

```
end if;
```

```
--in cazul in care stare numarare este true atunci numara secunde
```

```
if div10ms'event and div10ms='1' and stare_numarare = true then
```

```
    cnt_unit := cnt_unit+'1';
```

```
if cnt_unit = "1010" then
```

```
    cnt_unit := "0000";
```

```
    cnt_zeci := cnt_zeci + '1';
```

```
if cnt_zeci = "1010" then
```

```
    cnt_zeci := "0000";
```

```
    c_out := '1';
```

```
else
```

```
    c_out := '0';
```

```
end if;
```

```
else
```

```
    c_out := '0';
```

```

        end if;
    end if;
end if;
--actualizarea semnalelor de iesire
sutimi(3 downto 0) <= cnt_unit;
sutimi(7 downto 4) <= cnt_zeci;
sut_sec <= c_out;
end process;

--numarator secunde
process(sut_sec, ev_reset)
variable cnt_unit, cnt_zeci : std_logic_vector(3 downto 0);
variable c_out : std_logic;
begin
    --daca este reset apasat reseteaza contorul
    if ev_reset = '1' then
        cnt_unit := "0000";
        cnt_zeci := "0000";
        c_out := '0';
    else
        --daca nu testeaza eveniment pe intrarea sut_sec
        if sut_sec'event and sut_sec='1' then
            cnt_unit := cnt_unit+'1';
            if cnt_unit = "1010" then
                cnt_unit := "0000";
                cnt_zeci := cnt_zeci + '1';
                if cnt_zeci = "0110" then
                    cnt_zeci := "0000";
                    c_out := '1';
                else
                    c_out := '0';
                end if;
            else
                c_out := '0';
            end if;
        end if;
    end if;
end if;
--actualizare semnale
secunde(3 downto 0) <= cnt_unit;
secunde(7 downto 4) <= cnt_zeci;
sec_min <= c_out;
end process;

--numarator minute
process(sut_sec, ev_reset)
variable cnt_unit, cnt_zeci : std_logic_vector(3 downto 0);

```

```

begin
    --daca este reset apasat reseteaza contorul
    if ev_reset = '1' then
        cnt_unit := "0000";
        cnt_zeci := "0000";
    else
        --daca nu testeaza eveniment pe intrarea sut_sec
        if sut_sec'event and sut_sec='1' then
            cnt_unit := cnt_unit+'1';
            if cnt_unit = "1010" then
                cnt_unit := "0000";
                cnt_zeci := cnt_zeci + '1';
                if cnt_zeci = "1010" then
                    cnt_zeci := "0000";
                end if;
            end if;
        end if;
    end if;
    --actualizare semnale
    minute(3 downto 0) <= cnt_unit;
    minute(7 downto 4) <= cnt_zeci;
end process;

--afisare valori pe iesire
decodor_sutimi_low : decod7seg port map (sutimi(3 downto 0),data_out(6 downto 0));
decodor_sutimi_high : decod7seg port map (sutimi(7 downto 4),data_out(13 downto 7));
decodor_secunde_low : decod7seg port map (secunde(3 downto 0),data_out(20 downto 14));
decodor_secunde_high : decod7seg port map (secunde(7 downto 4),data_out(27 downto 21));
decodor_minute_low : decod7seg port map (minute(3 downto 0),data_out(34 downto 28));
decodor_minute_high : decod7seg port map (minute(7 downto 4),data_out(41 downto 35));
end crono_a;

```

Se va realiza implementarea pe macheta cu Spartan 3. Butoanele Start, stop și reset se vor asocia cu butoanele de pe macheta (SW_USER0...2), ieșirea cu afișajul 7 segmente (DIG0...5SEG0...6) iar ceasul global cu intrarea de ceas care provine de la oscilatorul extern de 50MHz aflată pe pinul AA12.

4. Temă

1. Să se descrie, utilizând limbajul VHDL, o structură logică care poate efectua operații logice pentru două numere pe 4 biți: SI logic și SAU logic.

2. Să se proiecteze un ceas. Ultimii doi digiți vor fi secunde, următorii minute și cei mai semnificativi ore.
- 3.

Criterii de evaluare:

Cerințe laborator	
1	3P
2	3P
Temă	
1	2P
2	2P